

NetComms 2 Module 4 Notes: Understanding and Using APIs

DevNet Associate, Module 4. Built from DEVASC_Module_4. The blue boxes are memory aids I made.

How To Use These Notes

Module 4 has eight topics: **4.1 Introducing APIs**, **4.2 Design Styles**, **4.3 Architectural Styles**, **4.4 REST APIs**, **4.5 Authentication**, **4.6 Rate Limits**, **4.7 Webhooks**, **4.8 Troubleshooting**. The biggest exam targets are the **HTTP methods**, **status codes**, **REST constraints**, and **authentication types**.

4.1 Introducing APIs

An **Application Programming Interface (API)** lets one piece of software **talk to another**. It uses common web-based interactions or protocols plus its own proprietary standards, and it decides **what data, services, and functionality** an application exposes to third parties, so an app can expose things **securely**.

Three uses: **automation** (scripts that do manual tasks), **data integration** (consume or react to another app's data), **functionality** (bring another app's function into your product).

Why popular: APIs are designed into modern products and well tested, and simple languages like **Python** let non-software engineers consume them.

Memory Aid

API uses: **A-D-F** = **A**utomate, **D**ata, **F**unction.

4.2 API Design Styles

	Synchronous	Asynchronous
Response	Replies directly with the data immediately	Replies with no data , just to say the request was received
When	Data for the request is readily available	The request takes time or data is not readily available
Client behavior	The client must wait for the response before running more code	The client keeps running ; the server-side design defines what the client must do later
Benefit	Immediate data, better performance if designed well	Application is not blocked , so better performance
Analogy on slide	Tickets sold first-come, first-served	Request accepted now, result later

Memory Aid

Sync = Stay and wait (you wait for the answer). **Async = Away, call me back.**

A product can have both, and each API's design is independent of the others.

4.3 API Architectural Styles

Three most popular: **RPC, SOAP, REST.**

RPC (Remote Procedure Call)

A **request-response** model that lets an application call a **procedure in another application**; the method runs and the result is returned. It is an API style that works over different transports:

XML-RPC, JSON-RPC, NFS (Network File System), and SOAP.

SOAP (Simple Object Access Protocol)

An **XML-based messaging protocol** for communication between applications on different platforms or built in different languages. SOAP is:

- **Independent:** any application can talk to any other, on any operating system
- **Extensible:** features such as reliability and security can be added
- **Neutral:** works over any protocol (HTTP, SMTP, TCP, UDP, JMS)

A SOAP message is an XML document with up to four elements:

Element	Contains
Envelope	The root element of the XML document
Header	Application-specific information such as authorization and SOAP attributes
Body	The data to be transported to the recipient
Fault	Error and/or status information

Memory Aid

SOAP is an **Envelope** holding a **Header, Body, and Fault: E-H-B-F**, "Every Hero Battles Fear."

Think of a letter: envelope outside, header is the address label, body is the letter, fault is the "return to sender" note.

REST (Representational State Transfer)

An architectural style authored by **Roy Thomas Fielding**, with **six constraints** that can be applied to any protocol.

Constraint	Meaning
Client-server	Client and server are independent of each other. The client can be

	built for many platforms, which simplifies the server.
Stateless	Each request must carry all the information the server needs . The server cannot hold session state .
Cache	Responses must say whether they are cacheable or not . If cacheable, the client may reuse the data for later requests.
Uniform interface	Four principles: identification of resources, manipulation of resources through representations, self-descriptive messages, hypermedia as the engine of application state .
Layered system	Hierarchical layers; each layer provides services only to the layer above it and uses services from the layer below.
Code-on-demand	The service can return executable code (for example JavaScript). This one is optional because running third-party code is a security risk.

Memory Aid

Sentence: **C**lients **S**tay **C**alm **U**nder **L**ayered **C**ode.

= **C**lient-server, **S**tateless, **C**ache, **U**niform interface, **L**ayered system, **C**ode-on-demand.

The **last one is the optional one** (code-on-demand).

Uniform interface four: **I-M-S-H** = Identify, Manipulate, Self-describe, Hypermedia.

4.4 Introduction To REST APIs

A REST API communicates over **HTTP** and uses the same concepts as HTTP: requests and responses, verbs, status codes, headers and body.

A REST Request Has Four Parts

Memory Aid

U-M-H-B: URI, Method, Header, Body.

1. URI (Uniform Resource Identifier)

Identifies the resource the client wants to manipulate (also called a URL). Its parts:

Part	Meaning
Scheme	Which HTTP protocol: http or https
Authority	Host and port
Path	The location of the resource on the server
Query	Extra details for scope, filtering, or clarifying the request

Memory Aid

S-A-P-Q: Say A Polite Query. In <https://example.com:443/books?author=x>: https = scheme, example.com:443 = authority, /books = path, ?author=x = query.

2. HTTP Method

Method	Action	Description
POST	Create	Create a new object or resource
GET	Read	Retrieve resource details
PUT	Update	Replace or update an existing resource
PATCH	Partial Update	Update some details of an existing resource
DELETE	Delete	Remove a resource

Memory Aid

CRUD order: Create = **POST**, Read = **GET**, Update = **PUT** (PATCH for a part), Delete = **DELETE**.

"**P**ost **G**ets **P**ut **D**own" = POST, GET, PUT, DELETE in CRUD order.

PUT = whole thing replaced. PATCH = patch a small hole.

3. Header

Name-value pairs separated by a colon: name: value. Two types:

Type	Purpose	Example
Request header	Extra information not related to the message content	Authorization: Basic ... (credentials)
Entity header	Describes the content of the body	Content-Type: application/json

		(format of the body)
--	--	----------------------

4. Body

Holds the data for the resource. **POST, PUT, and PATCH** usually include a body; the body is **optional depending on the method**. If a body is sent, its type **must be stated in the Content-Type header**.

A REST Response Has Three Parts

Memory Aid

S-H-B: Status, Header, Body.

HTTP Status Code Categories

Three digits. The **first digit is the category**; the other two are numbered in order.

Range	Category	Meaning
1xx	Informational	Request received, continuing; responses have no body
2xx	Success	Server received and accepted the request
3xx	Redirection	Client must take additional action
4xx	Client Error	Request has bad syntax or invalid input
5xx	Server Error	Server could not fulfill a valid request

Memory Aid

1 Info, 2 Good, 3 Go elsewhere, 4 Your fault, 5 Server's fault.

Common Status Codes

Code	Message	Meaning	Trick
200	OK	Success, usually with a body	Everything fine
201	Created	Resource was created	Think POST
202	Accepted	Accepted for processing, still in process	Async: "got it, working on it"
400	Bad Request	Not processed because of an error in the request (misspelled resource, JSON syntax)	Malformed
401	Unauthorized	No valid authentication credentials	Who are you?
403	Forbidden	Understood, but rejected ; credentials are known but not enough privilege	I know you, but no.
404	Not Found	The resource path was not found	Wrong path
407	Proxy Authentication Required	Like 401, but you must authenticate with the proxy first	Proxy wants ID

409	Conflict	Conflict with the current state of the resource (edit conflict); retry later may work	Two editors
415	Unsupported Media Type	Body format not supported (XML sent to a JSON-only server)	Wrong Content-Type
429	Too Many Requests	Rate limit exceeded	Slow down
500	Internal Server Error	Unexpected server condition	Server broke
501	Not Implemented	Server lacks the functionality	Not built
502	Bad Gateway	Gateway or proxy got an invalid response from upstream	Bad reply upstream
503	Service Unavailable	Overloaded or under maintenance	Busy or down
504	Gateway Timeout	Gateway or proxy got no timely response from upstream	No reply upstream

Watch Out

401 vs 403 is a favorite. **401** = authentication problem (check username, password, API key, token, URI). **403** = authentication is fine but **not authorized**; fix by using credentials with enough privileges.

502 vs 504: 502 = invalid reply; 504 = no reply in time.

Response Headers And Extras

- **Response headers** (extra info not about the content): **Set-Cookie** (server sends cookies) and **Cache-Control** (directives all caches must obey, for example max-age=3600, public).
- **Entity header** in a response: **Content-Type** (format of the body).
- **Pagination**: data is broken into chunks; usually a **query parameter** picks the page.
- **Compression**: for large data that cannot be paginated. Ask with the **Accept-Encoding** request header. Values: **gzip**, **compress**, **deflate**, **br**, **identity**, and the asterisk wildcard.
- **Sequence diagram** for an API has three sequences on the slide: **Create session** (HTTPS with credentials), **Get devices**, **Create device** (a POST).

4.5 Authenticating To A REST API

REST APIs need authentication so random users cannot access, create, update, or delete data. Some read-only APIs with nothing confidential need none.

	Authentication	Authorization
Question	Who are you?	What are you allowed to do?
Meaning	Proves the user's identity	Defines access : has permission to do the action on that resource
Slide analogy	Showing government ID or biometrics at the airport	Showing your ticket at a concert

Memory Aid

AuthN = Name (identity). AuthZ = Zone (what zones you can enter).

Authentication Mechanisms

Type	How It Works
Basic	Sends username:password separated by a colon and encoded in Base64
Bearer	Uses a bearer token , a string generated by an authentication server such as an Identity Service (IdS)
API Key	A unique alphanumeric string generated by the server and assigned to a user. Two types: public and private .

Memory Aid

B-B-A: Basic, Bearer, API key. Basic = password pair. Bearer = token. API key = assigned string.

Base64 is encoding, not encryption.

Authorization: OAuth

- **Open Authorization (OAuth)** combines authentication with authorization and was created to fix insecure authentication mechanisms.
- Two versions, **OAuth 1.0** and **OAuth 2.0**. **OAuth 2.0 is not backwards compatible.**
- OAuth 2.0 lets a **third-party app get limited access** to an HTTP service.
- The user gives credentials **directly to the authorization server** (an **Identity Provider, IdP, or Identity Service, IdS**) to obtain an **access token** to share with the app. The process of getting the token is called a **flow**.

4.6 API Rate Limits

A rate limit controls **how many requests** a user or app can make per unit of time. It helps to **avoid server overload, give better service to all users, and protect against Denial-of-Service (DoS) attacks.**

Four Algorithms

Algorithm	How It Works
Leaky Bucket	Requests go into a queue in the order received. They can arrive at any rate , but the server processes the queue at a fixed rate . If the queue is full, the request is rejected .
Token Bucket	Each user gets a set number of tokens per time increment. Each request spends one token . No token means the request is rejected. The client must track its tokens.
Fixed Window Counter	A fixed time window has a counter of allowed requests. When the limit is hit, all further requests in that window are rejected .
Sliding Window Counter	Allows a fixed number of requests in a set duration. For each new request the server counts requests made from the start of the window to now .

Memory Aid

Leaky = **steady drip** out of a bucket (fixed outflow). **Token** = **tickets**, spend one per request. **Fixed window** = **a clock that resets** at set times. **Sliding window** = **a moving window** that counts back from right now.

Knowing And Exceeding The Limit

Header Key	Meaning
X-Rate-Limit-Limit	Maximum requests in the time unit
X-Rate-Limit-Remaining	How many requests you can still make in this window
X-Rate-Limit-Reset	When the window resets

When you exceed the limit the server rejects the request and returns an HTTP error. Most common codes: **429 Too Many Requests** or **403 Forbidden**.

Memory Aid

Limit, Remaining, Reset: **L-R-R**. Max, Left, When-again.

4.7 Webhooks

A **webhook** is an **HTTP callback**, an **HTTP POST to a specified URL**, that notifies your app when an event happens. Apps do not need to **poll**. Webhooks are called **reverse APIs** because the app **subscribes** by registering with the webhook provider. **Multiple apps** can subscribe to one webhook server. Examples: Cisco DNA Center webhooks for network events; Webex Teams notifying you of new messages in a room.

To Consume A Webhook, The App Must:

1. Be **running at all times** to receive HTTP POST requests.
2. **Register a URI** with the webhook provider.

3. Handle the incoming notifications.

Memory Aid

Webhook = the pizza shop calls you when it is ready (you do not keep calling to ask, which is polling).

Run, Register, Handle: **R-R-H**.

4.8 Troubleshooting API Calls

Keep the **API reference guide** and the **authentication information** handy before you troubleshoot.

No Response And No Status Code

Side	Causes And Fixes
Client side	User error (mistyped URI); invalid URI (for example missing scheme); wrong domain name ; connectivity issues (proxy, firewall, VPN); SSL/invalid certificate (only with HTTPS, because of the SSL handshake). In a lab where certificates are not valid you can turn off verification with the verify parameter in the Python requests library. Real fix: find the root cause.
Server side	API server powered off , cabling problems , domain name changed , network down . You get a long silence then a traceback . Use a network capture tool and server logs if you have access. Cannot be fixed from the client; contact the server administrator.

Memory Aid

Client causes, **U-I-W-C-S**: User error, Invalid URI, Wrong domain, Connectivity, SSL certificate.

Steps When You Get A Status Code

1. **Check the return code** (print it during development).
2. **Check the response body** (print it too; the server often says what is wrong, for example "No id field provided").
3. Use a **status code reference** if that does not solve it.

The status code is part of **HTTP/1.1 (RFC 7231)**: the **first digit is the class**; the last two digits do not categorize anything.

Common 4xx Fixes

Code	Fix
400	Fix misspelled resources or JSON syntax problems; send required fields (example: the id)
401	Check username , password , API key , token , request URI ; add the authentication to the call
403	Not an authentication issue; the user lacks privileges. Use an account with enough privileges .
407	Authenticate with the proxy first
409	Conflict, such as two people editing; retrying later may work once the server resolves it
415	Send a format the server supports ; add or correct the Content-Type header such as application/json

Summary: 4xx = client-side errors, 5xx = server-side errors. Server-side problems are fixed by the server administrator.

One-Page Cram Sheet

Topic	Remember
API uses	Automation, data integration, functionality
Sync vs Async	Sync returns data now and client waits; Async returns no data first, client keeps going
Architectural styles	RPC, SOAP, REST
SOAP message	Envelope (root), Header, Body, Fault
REST author	Roy Fielding; six constraints; code-on-demand is optional
REST constraints	Client-server, Stateless, Cache, Uniform interface, Layered, Code-on-demand
Request parts	URI, Method, Header, Body
URI parts	Scheme, Authority (host and port), Path, Query
Methods	POST create, GET read, PUT update, PATCH partial, DELETE delete
Response parts	Status, Header, Body
Codes	200 OK, 201 Created, 202 Accepted, 400, 401, 403, 404, 500, 503; 429 too many requests
401 vs 403	401 who are you; 403 I know you, no
Auth types	Basic (Base64), Bearer (token), API key; OAuth does authorization
Rate limit algorithms	Leaky bucket, Token bucket, Fixed window, Sliding window
Rate limit headers	Limit, Remaining, Reset
Webhook	HTTP POST callback, reverse API, no polling; app must run, register, handle
Troubleshooting	Check code, check body, use reference; 4xx client, 5xx server